Version: Zurich

SERVICENOW APPLICATION DEVELOPER

Client-side Scripting

ARTICLE (1 OF 21)

Client-side Scripting Objectives

In this module you will learn to:

- Describe the purpose of a client-side script and give examples of what client-side scripts can do
- Create and test Client Scripts
- Create and test UI Policy scripts
- Use the *GlideForm* and *GlideUser* APIs in scripts
- Determine whether to use UI Policy scripts or Client Scripts

ARTICLE (2 OF 21)

About This Learning Module

> **IMPORTANT**: The content in this learning module was last updated for the ***San Diego*** ServiceNow release and was not updated for the ***Utah*** release. You may see differences between the ***Utah*** release and the content in this learning module.

Various examples are used on the concept pages in this learning module. You do not need to attempt to recreate the examples. You will develop the *NeedIt* application in the hands-on exercises.

Exercises are indicated in three ways:

- *Exercise* icon in the Navigation pane.
- *Exercise* icon and the word *Exercise* at the top of the page.
- The word *Exercise* or the word *Challenge* in the page title.

The *NeedIt* application allows users to request services from multiple departments. You will use source control to begin with all *NeedIt* application files needed for this learning module.

EXERCISE (3 OF 21)

Exercise: Fork Repository and Import Application for the Client-side Scripting Module

ServiceNow uses GitHub to provide application repositories to copy and use with the Developer Site learning content. The repositories contain tags, which are fixed sets of application files, to start you with a partially built application. By copying and importing a ServiceNow-provided repository into your Personal Developer Instance (PDI), you get all the files needed for the hands-on exercises in the modules.

> **NOTE**: See the GitHub Guide (/dev.do#!/guide/utah/now-platform/github-guide/github-and-the-developer-site-training-guide-introduction) for more information on how ServiceNow uses GitHub with the Developer Program learning content and to see a video on how to fork a repository and import an application.

In this exercise, you will:

1. Fork the ServiceNow repository to your GitHub account.
2. Import the application into your PDI from your fork of the repository.

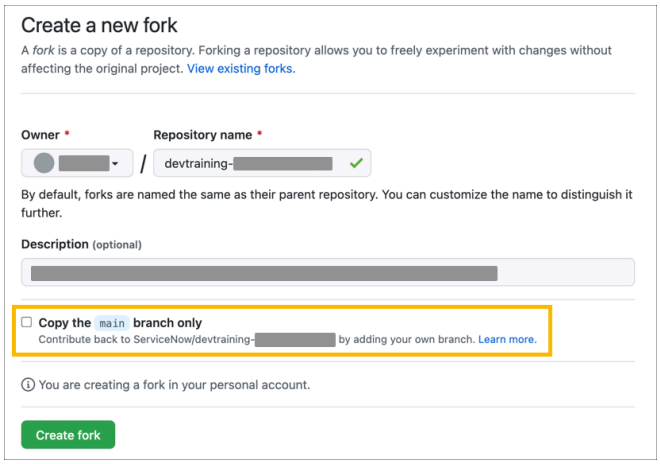
> **IMPORTANT**: If you have already forked and imported the repository, you can proceed to the next exercise, where you will create a branch from a tag to load the application files to your PDI. The *NeedIt* application files are needed to complete the module.

Fork the Repository

In this section of the exercise, you will create a personal fork of the application repository to use with Developer Site learning content.

1. In a web browser, open github.com (https://github.com/).
2. If you have a GitHub account, sign in. If not, sign up for a new account.
3. Once signed in, open the NeedIt repository (https://github.com/ServiceNow/devtraining-needit-utah).
4. Click the **Fork** button (

) to create a copy of the repository in your GitHub account.
5. On the *Create a new fork* page, deselect the **Copy the main branch only** option.



6. Select your personal GitHub account as the fork *Owner*, then click the **Create fork** button.

7. Verify the URL for your fork of the repository is similar to: *<YourGitHubUsername>/devtraining-application-release*.

8. Copy the forked repository's URL.

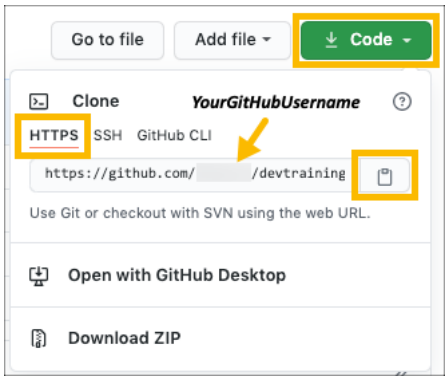
    1. Click the **Code** button.

    2. Make sure the URL contains your GitHub username, not ServiceNow.

    3. Make sure **HTTPS** is selected. If not, select the **HTTPS** tab in the *Clone* flyout.

    4. Click the **Copy to clipboard** button (

).



NOTE: You will use the copied URL to configure the connection to your forked repository in the next section.

Import the Application from the Forked Repository

In this section of the exercise, you will import the application repository into ServiceNow. As part of the process, you will first create a *Credential* record for your GitHub account, then you will use Studio to import the application repository into your PDI.

1. Log in to your PDI as the **admin** user. If you do not have a PDI, open the ServiceNow Developer Site (https://developer.servicenow.com) to obtain a **Utah** PDI.

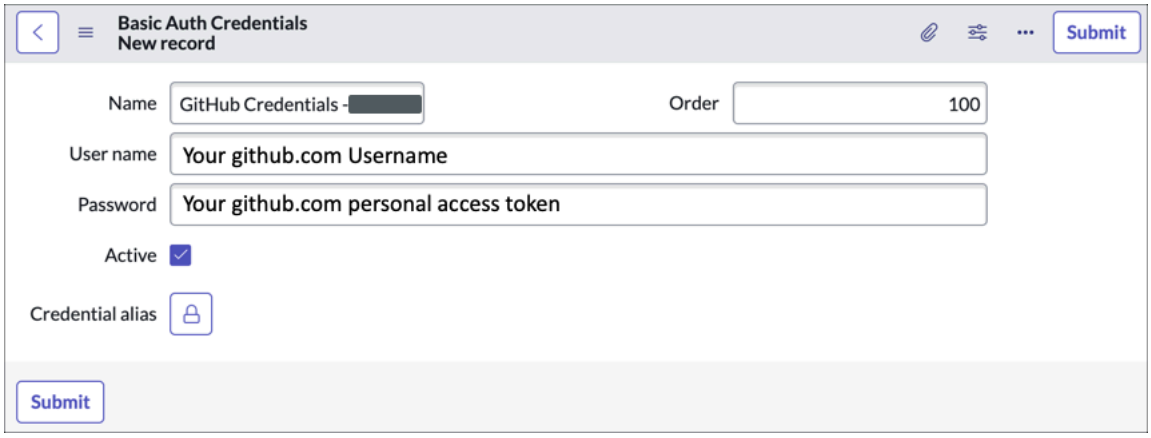
   NOTE: See the Personal Developer Instance (PDI) Guide (/dev.do#!/guide/utah/now-platform/pdi-guide/personal-developer-instance-guide-introduction) for instructions on how to obtain a PDI.

2. Create a *Credential* record for the GitHub connection.

   IMPORTANT: Credential records only need to be created once. If you have already created a credential record in another exercise, please skip this step.

   1. Use the *All* menu to open **Connections & Credentials > Credentials**.
   2. Click the **New** button.
   3. In the *What type of Credentials would you like to create?* list, click the **Basic Auth Credentials** link.
   4. Configure the *Credential* record.

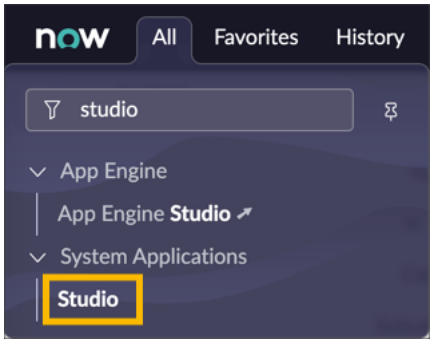
1. *Name*: **GitHub Credentials - <Your github.com Username>**
   User name: **<Your github.com Username>**
   Password: **<Your github.com personal access token>**



IMPORTANT: GitHub requires personal access tokens to access repositories from other platforms, like ServiceNow. **A personal access token is used in place of a password when authenticating**. See the Authenticating to GitHub (/dev.do#!/guides/utah/developer-program/github-guide/using-servicenow-provided-application-repositories#authenticating-to-github) section of the *GitHub Guide* for instructions on how to create a GitHub personal access token.

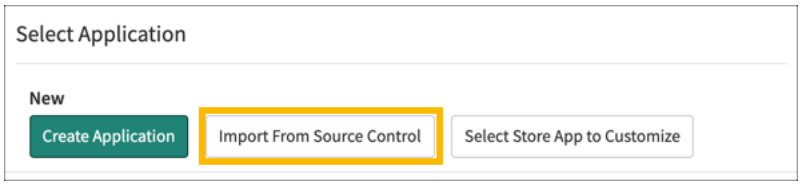
5. Click the **Submit** button.

3. Use the *All* menu to open **System Applications > Studio**.



4. Studio opens in a new browser tab.

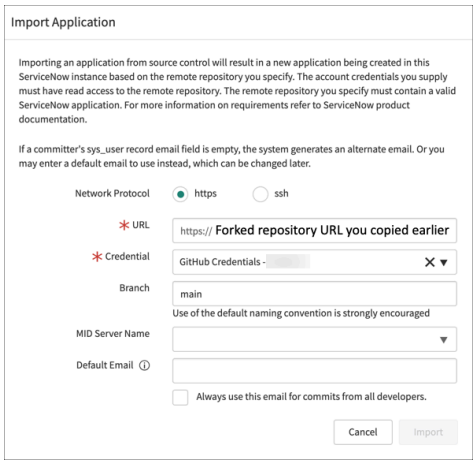
5. In the *Select Application* dialog, click the **Import From Source Control** button.



6. In the *Import Application* dialog, configure the connection to the forked repository.

   1. *URL*: **<URL you copied for your forked version of the repository>**
      Credential: **GitHub Credentials - <Your github.com Username>**
      Branch: **main**

   NOTE: When you change the *Branch* value to **main**, an information message informs you that *Use of the default naming convention is strongly encouraged*. The value in the *Branch* field must exist in the repository. The Developer Site training repositories all have a *main* branch, which should be used in place of the default value.



7. Click the **Import** button.

8. When the application import is complete, click the **Select Application** button.

   NOTE: If the connection fails, you may have entered the ServiceNow repository URL in the URL field instead of the forked repository URL, or you may have enabled two-factor authentication on your GitHub account. See Troubleshooting GitHub Issues (/dev.do#!/guide/utah/now-platform/github-guide/troubleshooting-github-issues) for instructions on how to

troubleshoot the connection.

9. In the *Select Application* dialog, click the application to open it for editing in Studio.

IMPORTANT: You will not see any application files in Studio until you successfully create a branch from a tag in the next exercise.

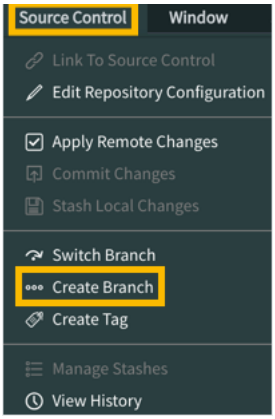
EXERCISE (4 OF 21)

Exercise: Create a Branch for the Client-side Scripting Module

In this exercise, you will create a branch of the *NeedIt* application for the Client-side Scripting module that includes the application files used in the module.

NOTE: Before you begin this exercise, you need to have forked and imported the *NeedIt* repository as described in *Exercise: Fork Repository and Import Application for the Client-side Scripting Module*.

1. If the *NeedIt* application is not open in Studio from the previous exercise, open it now.
    1. In the main ServiceNow browser window, use the *All* menu to open **System Applications > Studio**.
    2. In the *Select Application* dialog, click the **NeedIt** application.
2. In Studio, open the **Source Control** menu and select the **Create Branch** menu item.



3. Configure the branch.

1. *Branch Name*: **ClientScriptsModule**
   Create from Tag: **LoadForClientScriptsModule**

4. Click the **Create Branch** button.

5. Click the **Close** button.

6. To load the application files included in the tag, return to the main ServiceNow browser tab (not Studio) and click the browser's reload button to refresh the page.

> **NOTE**: If branch creation fails, you may have entered the ServiceNow repository URL in the URL field instead of the forked repository URL or enabled two-factor authentication on your GitHub account. See the Troubleshooting GitHub Issues (/dev.do#!/guide/utah/now-platform/github-guide/troubleshooting-github-issues) section of the *GitHub Guide* for instructions on how to troubleshoot GitHub connection issues.

ARTICLE (5 OF 21)

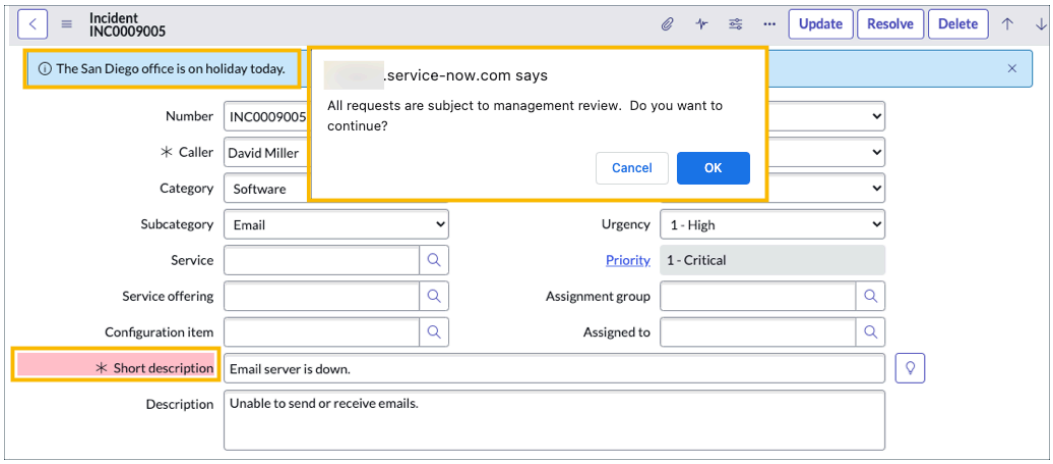
Introduction to Client-side Scripting

Scripts in ServiceNow fall into two categories:

- Client-side
- Server-side

This learning module is about client-side scripting. Client-side scripts execute within a user's browser and are used to manage forms and form fields. Examples of things client-side scripts can do include:

- Place the cursor in a form field on form load
- Generate alerts, confirmations, and messages
- Populate a form field in response to another field's value
- Highlight a form field
- Validate form data
- Modify choice list options
- Hide/Show fields or sections



In this module you will learn to write, test and debug two types of client-side scripts:

- Client Scripts
- UI Policy Scripts

ARTICLE (6 OF 21)

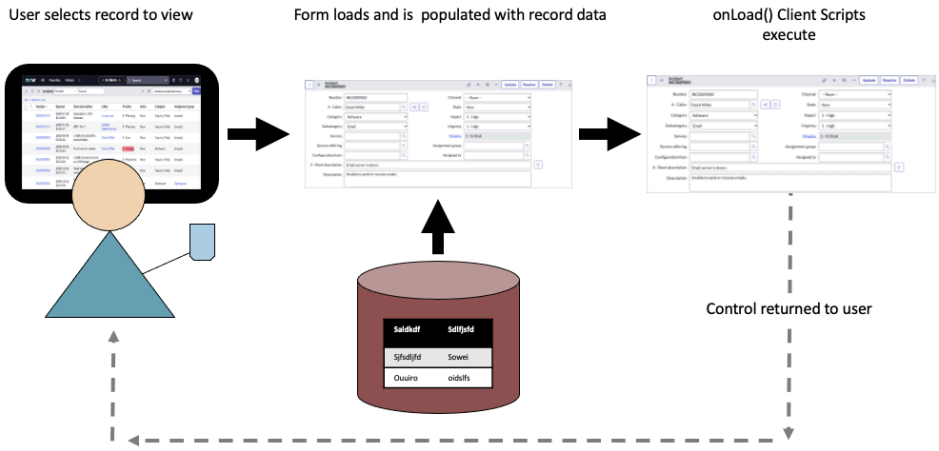
Client Script Types

A Client Script executes client-side script logic when forms are:

- Loaded
- Changed
- Submitted

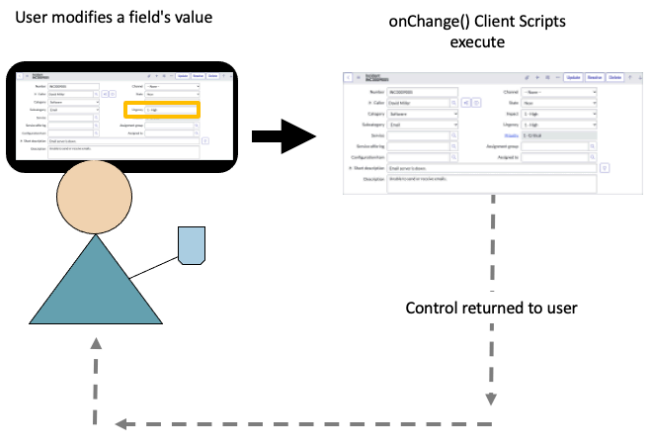
onLoad

onLoad Client Scripts execute script logic when forms are loaded. Use *onLoad* Client Scripts to manipulate a form's appearance or content. For example, setting field or form-level messages based on the presence of a value. Use *onLoad* Client Scripts sparingly as they impact form load times.



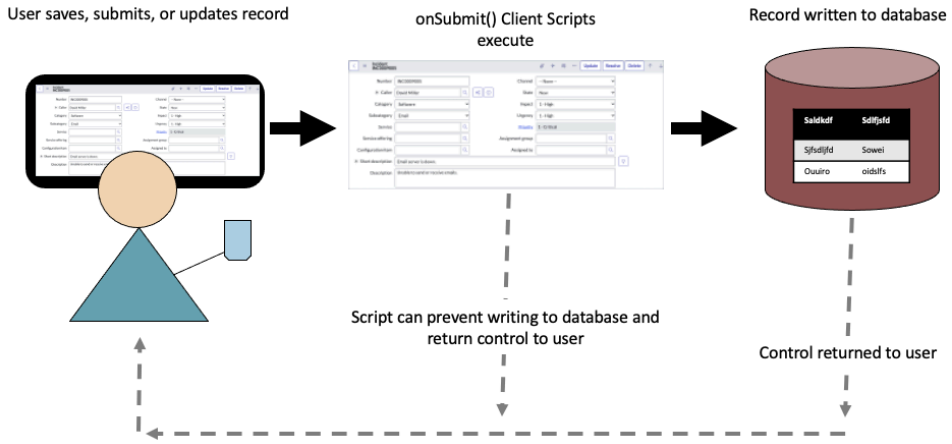
onChange

onChange Client Scripts execute script logic when a particular field's value changes. Use *onChange* Client Scripts to respond to field values of interest and to modify another field's value or attributes. For example, if the *State* field's value changes to *Closed Complete*, generate an alert and make the *Description* field mandatory.



onSubmit

onSubmit Client Scripts execute script logic when a form is submitted. Use *onSubmit* Client Scripts to validate field values. For example, if a user submits a Priority 1 record, the script can generate a confirmation dialog notifying the user that the executive staff are copied on all Priority 1 requests.



NOTE: The *Type* field has a fourth option: *onCellEdit*. The *onCellEdit* Client Script type is for lists rather than forms. *onCellEdit* Client Scripts are not addressed in this learning module.

ARTICLE (7 OF 21)

Creating Client Scripts

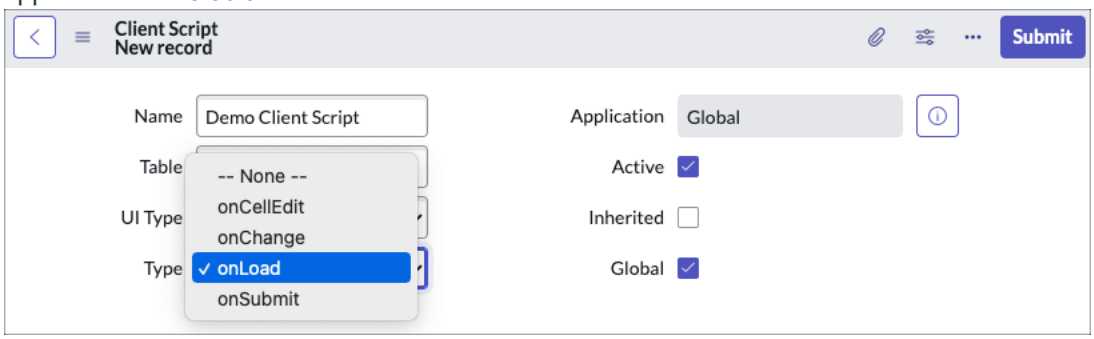
The procedure for adding files to an application in Studio is the same regardless of file type:

1. Click the **Create Application File** link.
2. Choose the new file type, in this case, **Client Script**.
3. Configure the new file.

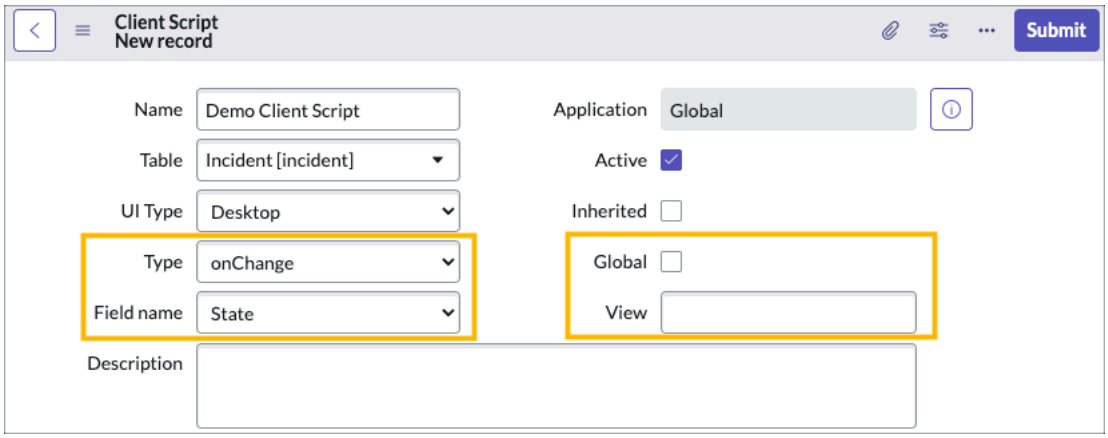
Configuring the Client Script

As with any script, the configuration tells the script when to execute. The Client Script configuration options are:

- ***Name***: Name of Client Script. Use a standard naming scheme to identify custom scripts.
- ***Table***: Table to which the script applies.
- ***UI Type***: Select whether the script executes for *Desktop and Tablet* or *Mobile/Service Portal* or *All*.
- ***Type***: Select when the script runs: *onChange*, *onLoad*, or *onSubmit*.
- ***Field Name***: Used only if the script responds to a field value change (*onChange*); name of the field to which the script applies.
- ***Active***: Controls whether the script is enabled. Inactive scripts do not execute.
- ***Inherited***: If selected, this script applies to the specified table and all tables that inherit from it. For example, a client script on the *Task* table will also apply to the *Change*, *Incident*, *Problem* and all other tables which extend *Task*.
- ***Global***: If *Global* is selected the script applies to all Views. If the *Global* field is not selected you must specify the View.
- ***View***: Specifies the View to which the script applies. The *View* field is visible when *Global* is not selected. A script can only act on fields that are part of the selected form View. If the *View* field is blank the script applies to the *Default* view.



The *Field name* field is available for onChange Client Scripts. The *View* field is available when the *Global* option is not selected.



DEVELOPER TIP: Whenever client-side logic is modified, reload the main ServiceNow browser window to ensure the latest logic is loaded.

ARTICLE (8 OF 21)

The Script Field

When the type value is set, a script template is automatically inserted into the *Script* field.

onLoad Script Template

The *onLoad* script template:

The *onLoad* function has no arguments passed to it. As indicated by the comment, add your script logic in the *onLoad* function. It is a best practice to document your code so include comments to explain what the script does. Your future self will thank you for the clearly documented script.

This example script generates an alert when a user requests a form load for a record. The user cannot interact with a form until *onLoad* Client Scripts complete execution.

Script

```
function onLoad() {
    //Type appropriate comment here, and begin script below
    alert("Thank you for loading the form.  When you close this alert you will be able to interact with the form.");
}
```

onSubmit Script Template

The *onSubmit* script template:

Script

```
function onSubmit() {
   //Type appropriate comment here, and begin script below

}
```

The *onSubmit* function has no arguments passed to it. As indicated by the comment, add your script logic in the *onSubmit* function.

This example script generates an alert when a user saves a *NeedIt* record. The record is not submitted to the server until the *onSubmit* Client Scripts complete execution and return true.

Script

```
function onSubmit() {
   //Generate an alert to thank the user for submitting a record
    alert("Thank you for submitting a record.");
}
```

onChange Script Template

The *onChange* Script template:

Script

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
   if (isLoading || newValue === '') {
      return;
   }

   //Type appropriate comment here, and begin script below

}
```

The *onChange* function is automatically passed five parameters by ServiceNow. Although you do not need to do anything to pass the parameters, you can use them in your script.

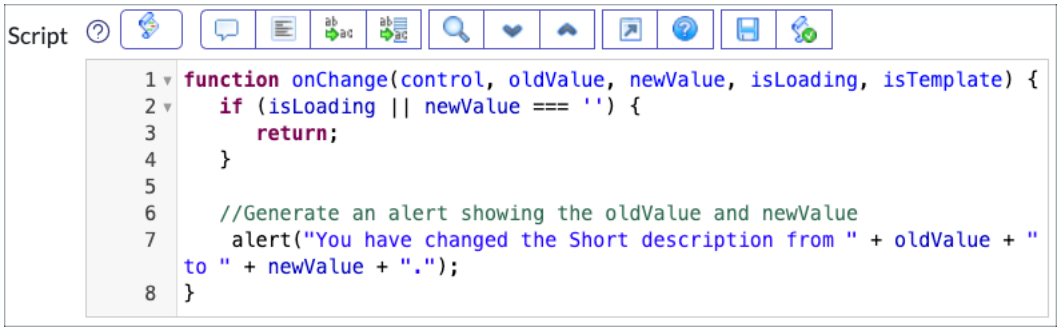
- ***control***: field the Client Script is configured for.
- ***oldValue***: value of the field when the form loaded and prior to the change.
- ***newValue***: value of the field after the change.
- ***isLoading***: boolean value indicating whether the change is occurring as part of a form load. Value is *true* if change is due to a form load. When forms load, all the field values on the form change as the record is loaded into the form.
- ***isTemplate***: boolean value indicating whether the change occurred due to population of the field by a template. Value is *true* if change is due to population from a template.

When a user selects a record to load, the form and form layout are rendered first and then the fields are populated with values from the database. From the technical perspective, all field values are changed, from nothing to the record's values, when a form loads. The *if* statement in the template assumes that *onChange* Client scripts should not execute their script logic when a form loads. The *onChange* Client Script also stops execution if the *newValue* for a field is no value. Depending on your use case, you can modify or even remove the *if* statement. For example:

```
//Stop script execution if the field value change was caused by a Template
          if(isLoading || newValue === '' || isTemplate) {
             return;
          }
```

For this example, the *onChange* Client Script executes because of changes to the *Short description* field value:

When a user changes the value of the *Short description* field *on the form* the *onChange* script logic executes. The example generates an alert stating that the value of the *Short description* field has changed from the value the field had when the form loaded to the new value on the form.



The value of the *Short description* field has changed *only* on the form. Changes are not sent to the database until a user saves, updates, or submits the record.

It is important to know that the value of *oldValue* is set *when the form loads*. Regardless of how many times the value of the *Short description* field changes, *oldValue* remains the same until the form is re-loaded from the database.

1. Form loads:
    1. *oldValue* = hello
    2. *newValue* has no value
2. User changes the value in the *Short description* field to bye:
    1. *oldValue* = hello
    2. *newValue* = bye
3. User changes the value in the *Short description* field to greetings:
    1. *oldValue* = hello
    2. *newValue* = greetings
4. User saves the form and reloads the form page:
    1. *oldValue* = greetings
    2. *newValue* has no value

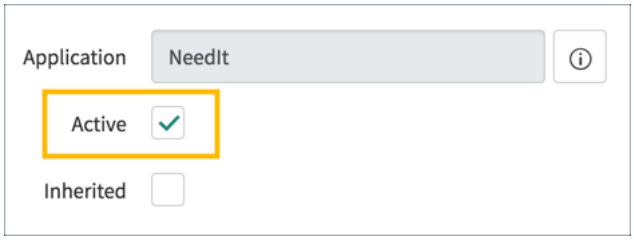
EXERCISE (9 OF 21)

Exercise: Test Client Script Types

In this exercise you will test the *onLoad*, *onChange*, and *onSubmit* Client Script types using pre-built Client Scripts from the GitHub repository you forked.

Preparation

1. If the *NeedIt* application is not open in Studio from the last exercise, open it now.
    1. In the main ServiceNow browser window, open the **All** menu.
    2. In the *Filter* field, enter **Studio**.
    3. Open **System Applications > Studio**.
    4. In the *Select Application* dialog, click the **NeedIt** application.
2. Make the *NeedIt onLoad Example* Client Script active.
    1. In Studio, use the Application Explorer to open **Client Development > Client Scripts > NeedIt onLoad Example**.
    2. Select the **Active** option.

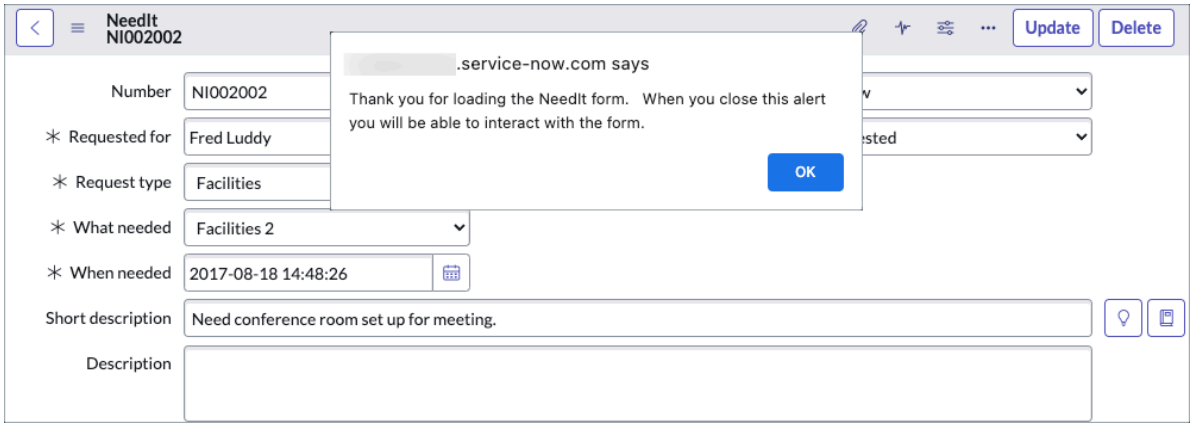


    3. Examine the configuration to see which table the script is for.
3. Examine the *Script* field to determine what the script does.
4. Click the **Update** button.

5. Using the same strategy, make the **NeedIt onSubmit Example** and the **NeedIt onChange Example** Client Scripts active. For the onChange Client Script, make a note of the value in the *Field name* field.

Testing the Client Scripts

1. In the main ServiceNow browser window (not Studio) use the *All* menu to open **NeedIt > All**.
2. Test the *NeedIt onLoad Example* Client Script.
   1. Open the *NeedIt* record of your choice for editing.
   2. Does an alert appear? If not, make sure you made the *NeedIt onLoad Example Client Script* active and saved the change.



   3. With the alert still open, try to modify a field value on the form. You should not be able to modify the form until the *NeedIt onLoad Example* script finishes execution. *onLoad* Client scripts with alerts prevent control of the form from being given to the user until the alert is closed.
   4. Click the **OK** button to close the alert.
3. Test the *NeedIt onChange Example* Client Script.
   1. Edit any field on the form *except* the *Short description* field.
   2. No alert should appear.
   3. Edit the value in the *Short description* field then click on the form outside of the field.
   4. An alert should appear showing the *oldValue* and the *newValue*. If no alert appears, make sure you made the *NeedIt onChange Example* Client Script active and saved the change.
   5. Click the **OK** button to close the alert.
   6. Edit the *Short description* field again then click outside of the field.
   7. In the **alert**, notice that *oldValue* has not changed.
   8. Click the **OK** button to close the alert.
4. Test the *NeedIt onSubmit Example* Client Script.
   1. Click the **Update** button on the form header.
   2. Does the alert appear? If not, make sure you made the *NeedIt onSubmit Example* Client Script active and saved the change.
   3. Click the **OK** button to close the alert.
5. Return to Studio and make the Client Scripts inactive:

- *NeedIt onLoad Example*
- *NeedIt onChange Example*

- *NeedIt onSubmit Example*

ARTICLE (10 OF 21)

The GlideForm (g_form) Class

The *GlideForm* (https://developer.servicenow.com/dev.do#!/reference/api/utah/client/c_GlideFormAPI) client-side API provides methods for managing form and form fields including methods to:

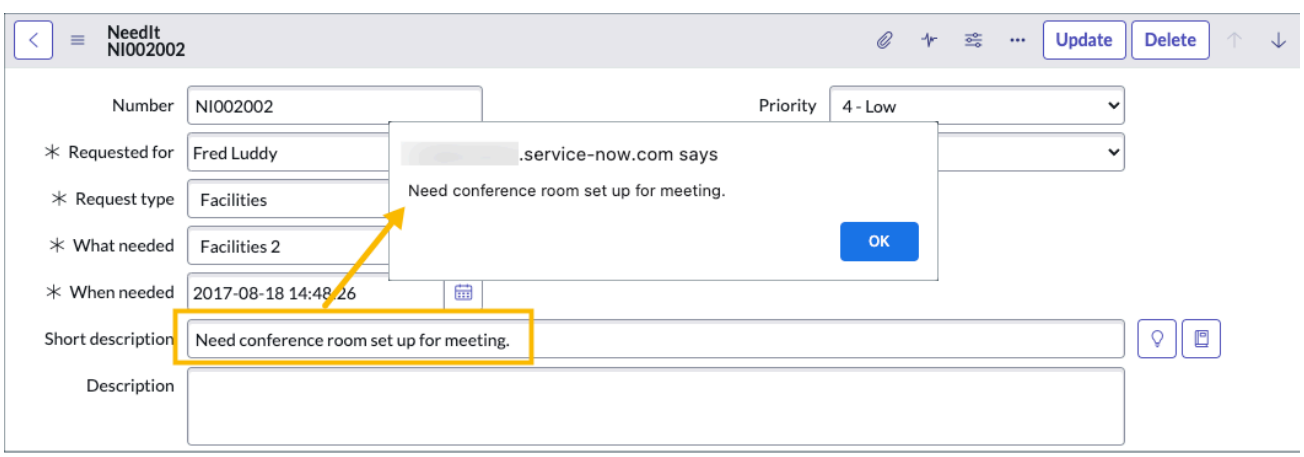
- Retrieve a field value on a form
- Hide a field
- Make a field read-only
- Write a message on a form or a field
- Add fields to a choice list
- Remove fields from a choice list

The *GlideForm* methods are accessed through the global *g_form* object that is only available in client-side scripts. To use methods from the *GlideForm* class use the syntax:

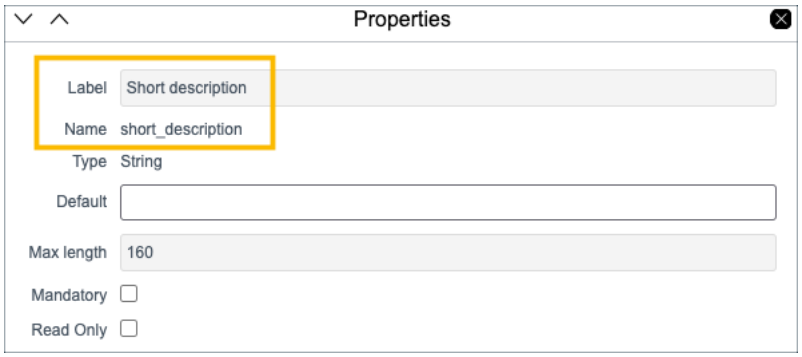
```
g_form.<method name>
```

For example, the *GlideForm* API has a method called *getValue()*. The *getValue* method gets the value of a field on the form (not the value in the database). The example script gets the value of the *Short description* field from the form and displays the *Short description* field value in an alert.

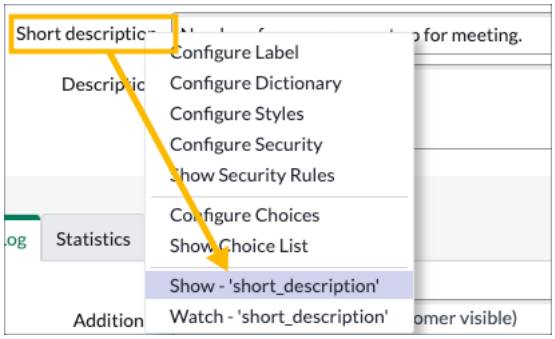
```
alert(g_form.getValue('short_description'));
```



Notice that on the form, the field name is *Short description* but the script refers to the same field as **short_description**. Why are these two values different? All table fields have both a *Label* and *Name*. The *Label* is the human friendly version of the field name which appears on forms. The *Name* is the value used to reference a field in a script. *Names* are always lowercase and never contain spaces.



There are a number of ways to find the *Name* and *Label* values for a field such as looking at the table definition, looking at the properties in Form Designer, or looking in the Dictionary. The fastest way to find the *Name* and *Label* values when a form is open is to right-click the field's *Label* in the form. The *Label* appears on the form and the *Name* appears in the right-click menu.



In addition to the *getValue*() method, other commonly used *GlideForm* methods include:

- *addOption()*
- *clearOptions()*
- *addInfoMessage()*
- *addErrorMessage()*
- *showFieldMsg()*
- *clearMessages()*
- *getSections()*
- *getSectionName()*

For a complete list of client-side APIs, visit the Developer Site API Reference (https://developer.service-now.com/dev.do#!/reference/api/utah/client/).

ARTICLE (11 OF 21)

The GlideUser (g_user) Class

The *GlideUser* (https://developer.servicenow.com/dev.do#!/reference/api/utah/client/c_GlideUserAPI) API provides methods and non-method properties for finding information about the currently logged in user and their roles. The typical use cases are personalizing feedback to the user and inspecting user roles. Note that client-side validation in any web application is easily bypassed.

The *GlideUser* API has properties and methods to:

- Retrieve the user's:
  - First name
  - Full name
  - Last name
  - User ID
  - User name
- Determine if a user has a particular role assigned

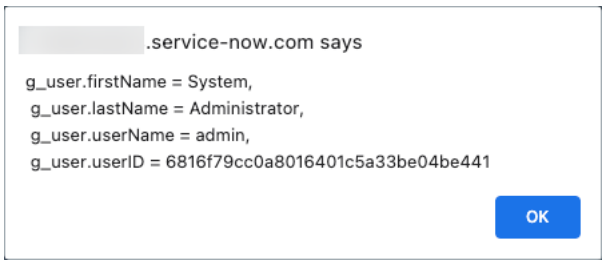
The *GlideUser* methods and non-method properties are accessed through the global *g_user* object which is only available in client-side scripts. To use methods and properties from the *GlideUser* class use the syntax:

```
g_user.<method or property name>
```

For example, the *GlideUser* API has a property called *userName*. The *userName* property's value is the currently logged in user's user name. The example script shows the difference between the *firstName*, *lastName*, *userName*, and *userID* property values.

```
alert("g_user.firstName = " + g_user.firstName
          + ", \n g_user.lastName = " + g_user.lastName
          + ", \n g_user.userName = " + g_user.userName
          + ", \n g_user.userID = " + g_user.userID);
```

The alert generated by the script is:



The *g_user.userID* property contains the record's *sys_id*. Every record has a 32-character unique *sys_id*.

Although you could concatenate the output of *g_user.firstName* with *g_user.lastName*, the convenience method, *g_user.getFullName()* concatenates the two.

The *GlideUser* API also has methods for determining if a user has a specific role. For example:

```
g_user.hasRole('client_script_admin');
```

The example script checks to see if the currently logged in user has the capability to create and edit Client Scripts (*client_script_admin* role). Note that the script returns true not only when the currently logged in user has the role assigned but also if the currently logged in user has the *admin* role. The *admin* user has all roles implicitly assigned. To test whether the currently logged in user has the role explicitly assigned, use the *hasRoleExactly()* method:

```
g_user.hasRoleExactly('client_script_admin');
```

For a complete list of client-side APIs, visit the Developer Site API Reference (https://developer.service-now.com/dev.do#!/reference/api/utah/client/).

EXERCISE (12 OF 21)

Exercise: Create Client Scripts

In this exercise you will write and test two Client Scripts:

- Script to set the **What needed** choice list values based on the value in the **Request type** field
- Set the **Requested for** value to the currently logged in user

Create the NeedIt Request Type Options Client Script

1. Examine the **Request type** and **What needed** fields on the NeedIt form.
    1. In the main ServiceNow browser window (not Studio), use the *All* menu to open **NeedIt > All**.
    2. Open the *NeedIt* record of your choice for editing.
    3. Click the **Request type** field to see the choices. You should see: *Facilities*, *Human Resources*, and *Legal*.
    4. Click the **What needed** field to see the choices. Notice the choices include departments other than the department selected in the *Request type* field.
    5. Use the *All* menu to open **NeedIt > Create New**.
    6. Notice the default value for the *Request type* field.
2. Return to Studio and create a Client Script.
    1. In Studio, click the **Create Application File** link.
    2. In the *Filter...* field enter the text **Client** OR select **Client Development** from the categories in the left hand pane.
    3. Select **Client Script** in the middle pane as the file type, then click the **Create** button.

NOTE: If you have annotations enabled, you will see an annotation:

New client-scripts are run in strict mode, with direct DOM access disabled. Access to jQuery, prototype and the window object are likewise disabled. To disable this on a per-script basis, configure this form and add the "Isolate script" field. To disable this feature for all new globally-scoped client-side scripts set the system property "glide.script.block.client.globals" to false.

The Client Scripts used in this module do not access the objects described in the annotation. You can ignore the message.

3. Configure the Client Script:

   1. *Name*: **NeedIt Request Type Options**
      Table: **NeedIt [x_<your_company_code>_needit_needit]**
      UI Type: **All**
      Type: **onChange**
      Field name: **Request type**
      Description: **Only display What needed choices that match the Request type value.**

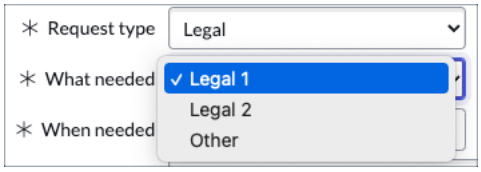
4. Replace the contents of the *Script* field with this script:

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
                    if ( newValue == '') {
                              return;
                    }

                    var whatneeded = g_form.getValue('u_what_needed');

                    // Clear all of the choices from the What needed field choice list
                    g_form.clearOptions('u_what_needed');

                    // If the value of the Request type field is hr, add
                    // two hr choices and other to the What needed field choice list
                    if(newValue == 'hr'){
                              g_form.addOption('u_what_needed','hr1','Human Resources 1');
                              g_form.addOption('u_what_needed','hr2','Human Resources 2');
                              g_form.addOption('u_what_needed','other','Other');
                    }
                    // If the value of the Request type field is facilities, add
                    // two facilities choices and other to the What needed field
                    // choice list
                    if(newValue == 'facilities'){
                              g_form.addOption('u_what_needed','facilities1','Facilities 1');
                              g_form.addOption('u_what_needed','facilities2','Facilities 2');
                              g_form.addOption('u_what_needed','other','Other');
                    }
                    // If the value of the Request type field is legal, add
                    // two legal choices and other to the What needed field
                    // choice list
                    if(newValue == 'legal'){
                              g_form.addOption('u_what_needed','legal1','Legal 1');
                              g_form.addOption('u_what_needed','legal2','Legal 2');
                              g_form.addOption('u_what_needed','other','Other');
                    }

                    // If the form is loading and it is not a new record, set the u_what_needed value to the
                    // value from the record before it was loaded
                    if(isLoading && !g_form.isNewRecord()){
                              g_form.setValue('u_what_needed', whatneeded);
                    }
          }
```

5. Click the **Submit** button.

Test the NeedIt Request Type Options Client Script

1. In the main ServiceNow browser window (not Studio) use the *All* menu to open **NeedIt > All**.
2. Open the record of your choice for editing. If you already have a *NeedIt* record open, reload the form for the changes to take effect.
3. Notice the value in the *Request type* field.
4. Click in the **What needed** field to open the choice list. The choices should match the value in the *Request type* field plus *Other*. For example:



5. Select another choice in the *Request type* field and verify that the *What needed* choices change too.

Create the NeedIt Set Requested for Client Script

1. Return to Studio and create a Client Script.
    1. In Studio, click the **Create Application File** link.
    2. In the *Filter...* field enter the text **Client** OR select **Client Development** from the categories in the left hand pane.
    3. Select **Client Script** in the middle pane as the file type, then click the **Create** button.
2. Configure the Client Script:
    1. *Name*: **NeedIt Set Requested for**
       Table: **NeedIt [x_<your_company_code>_needit_needit]**
       UI Type: **All**
       Type: **onLoad**
       Description: **Set the Requested for to the currently logged in user for new records. Users can change the field value**
3. Replace the contents of the *Script* field with this script:

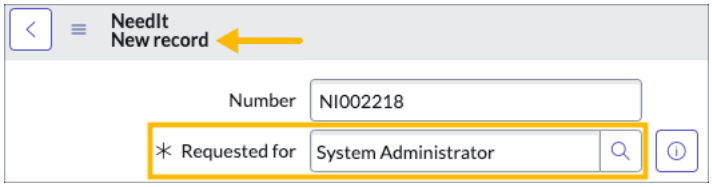
```
function onLoad() {
  //Check to see if the form is for a new record.  If it is a new record,
  //set the Requested for value to the currently logged in user.

    if(g_form.isNewRecord()){
        g_form.setValue('u_requested_for',g_user.userID);
    }
}
```

4. Click the **Submit** button.

Test the NeedIt Set Requested for Client Script

1. In the main ServiceNow browser window (not Studio) use the *All* menu to open **NeedIt > Create New**.

2. Examine the *Requested for* field. The value should automatically be set to *System Administrator*.



3. Change the *Requested for* value to **Beth Anglin**.

4. Add the values of your choice to the mandatory fields (indicated with an * on the form).

5. Click the **Submit** button.

6. From the list of records, open the record you just created.

7. Examine the value of the *Requested for* field.

ARTICLE (13 OF 21)

UI Policies

Like Client Scripts, UI Policies are client-side logic that governs form and form field behavior. Unlike Client Scripts, UI Policies do not always require scripting.

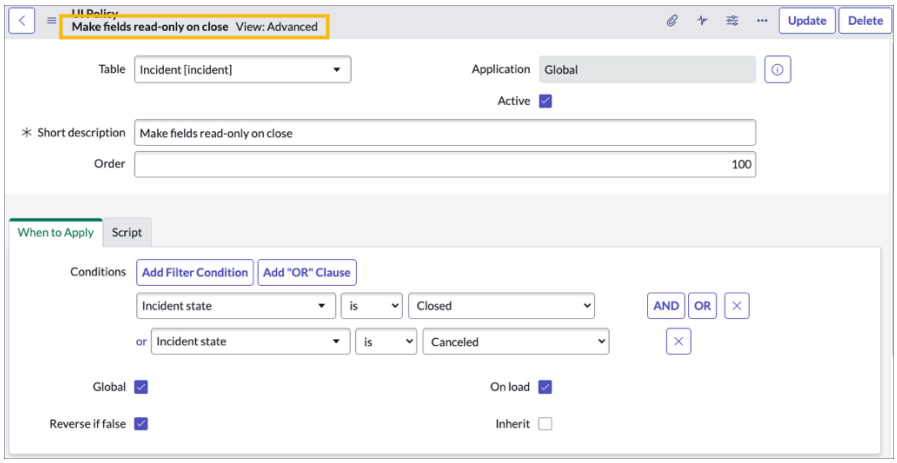
Creating UI Policies

The procedure for adding files to an application in Studio is the same regardless of file type:

1. Click the **Create Application File** link.
2. Choose the file type, in this case, **UI Policy**.
3. Configure the new file.

UI Policy Configuration

UI Policies have two views: *Default* and *Advanced*. The fields in the UI Policy configuration are different depending on which View is selected. The *Advanced* view displays all of the configuration fields. The *Default* view displays a subset of the fields.



- ***Table***: Form (table) to which the UI Policy applies.
- ***Application***: Identifies the scope of the UI Policy.
- ***Active***: Controls whether or not the UI Policy is enabled.
- ***Short description***: A short explanation of what the UI Policy does.
- ***Order***: If multiple UI Policies exist for the same table, use the *Order* field to set the order of evaluation of the UI Policy Conditions.
- ***Condition***: The condition(s) that must be met to trigger the UI Policy logic.
- ***Global***: If *Global* is selected the script applies to all views for the table. If the *Global* field is not selected you must specify the view.
- ***View***: Specifies the view to which the script applies. The *View* field is only visible when *Global* is not selected. A script can only act on fields that are part of the selected form view. If the *View* field is blank the script applies to the *Default* view.
- ***On load***: When selected, the UI Policy condition field is evaluated when a form loads in addition to when field values change. When not selected, the UI Policy Condition is evaluated only when field values change.
- ***Reverse if false***: Take the opposite action when the *Condition* field evaluates to *false*.
- ***Inherit***: When selected, executes the script for forms whose table is extended from the UI Policy's table.

DEVELOPER TIP: Enter a descriptive value in the *Short description* field because UI Policies do not have a *Name* field. When debugging, identify UI Policies by the *Short description* field value.

If the *Condition* field does not have a value, the condition returns true and the UI Policy logic will execute every time there is a change to a field value on the form.

The *Order* field sets the order of evaluation of UI Policy conditions for UI Policies for the same table. The order of evaluation is from the lowest number to the highest number (ascending order). By convention, *Order* field values are in round values of one hundred: 100, 200, 300 etc. This is not required.

> **DEVELOPER TIP**: Avoid ordering UI Policies as 1, 2, 3, etc. Leave a gap between *Order* field values to make it possible to insert a new UI Policy into the existing line-up without re-ordering the existing UI Policies.

ARTICLE (14 OF 21)

UI Policy Actions

UI Policy Actions are client-side logic in a UI Policy used to set three field attributes:

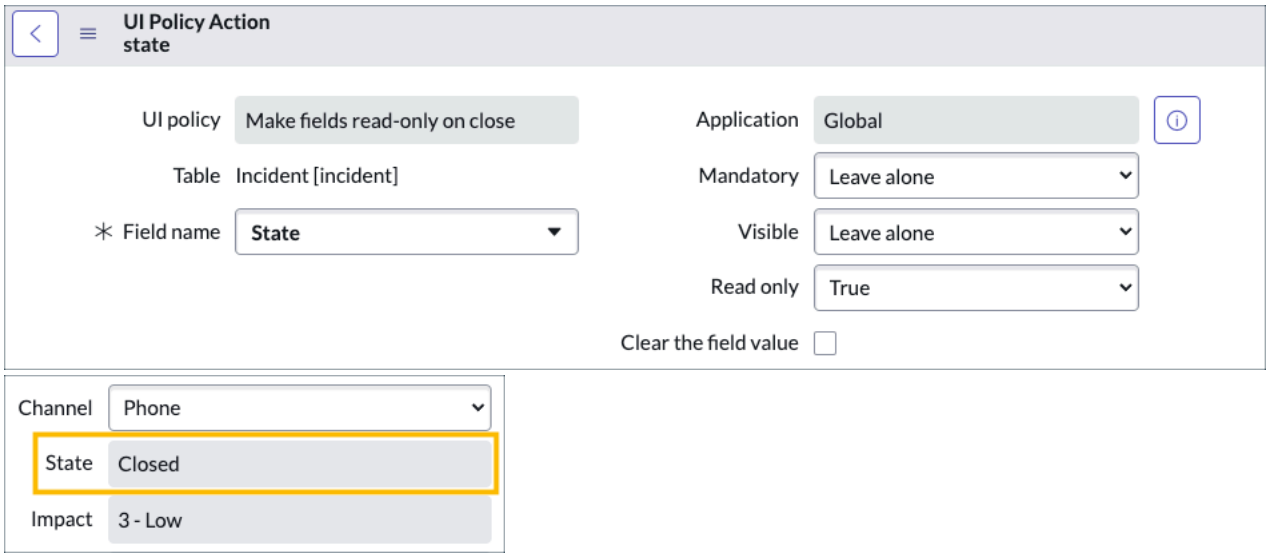
- *Mandatory*
- *Visible*
- *Read only*

Although you can use scripts to set these attributes using the *GlideForm (g_form)* API, UI Policy Actions do NOT require scripting to set the field attributes.

Creating UI Policy Actions

1. In Studio, create a UI Policy or open an existing UI Policy for editing.
2. Scroll to the *UI Policy Actions* related list. If creating a new UI Policy, the UI Policy must be saved before the *UI Policy Actions* related list is visible.
3. Click the **New** button.
4. Configure the UI Policy Action.
    1. Select a *Field* name.
    2. Set the *Mandatory*, *Visible*, or *Read-only* field values.
    3. ***True***: Apply the attribute to the field.
       False: Do not apply the attribute to the field.
       Leave alone: The attribute does not apply to the field.
5. To clear any existing value from the field, select the **Clear the field value** option.
6. Click the **Submit** button.

When the UI Policy condition tests *true*, the UI Policy Actions are applied. In the example, the *State* field is *Read only*. The *Mandatory* and *Visible* attribute values are not changed by the UI Policy Action.



What happens when the UI Policy condition tests *false*? There are two possible outcomes:

- No action is taken
- The opposite action is taken

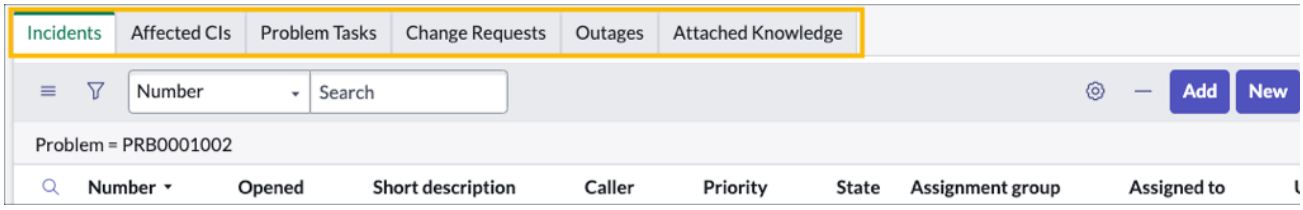
How does ServiceNow know which to do? The decision is made by the *Reverse if false* option in the UI Policy trigger.

- If *Reverse if false* is selected (default), the opposite action is taken in the UI Policy Actions. If a field is mandatory (true), the field will no longer be mandatory (false). That is to say, attributes which were true become false, and false become true. There are no changes to attributes which are set to Leave alone.
- If *Reverse if false* is not selected, no UI Policy Action logic is applied.

Creating UI Policy Related List Actions

Use UI Policy Related List Actions to show or hide related lists. The *Problem* form has multiple related lists:

- *Incidents*
- Affected CIs
- *Problem Tasks*
- *Change Requests*
- *Outages*
- *Attached Knowledge*



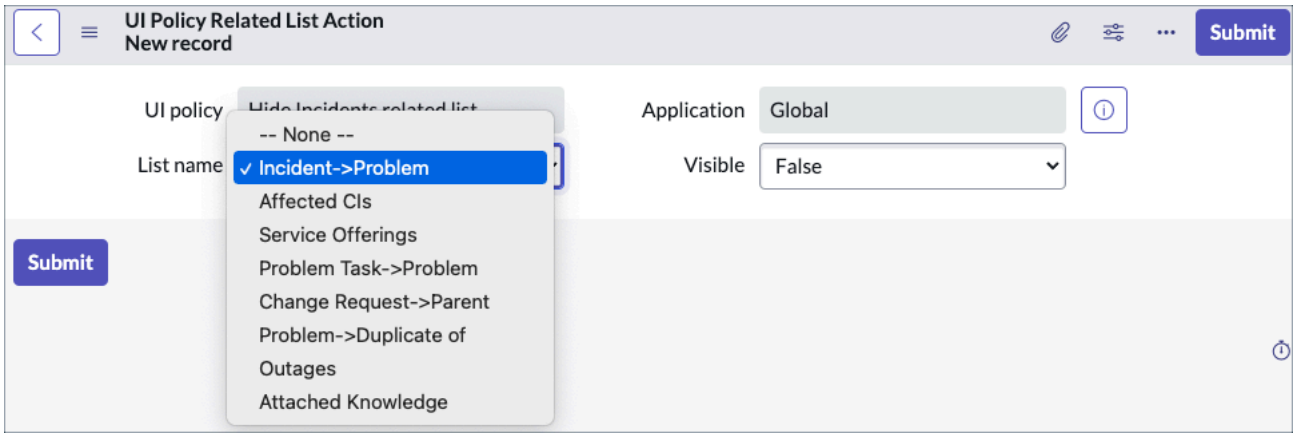
1. In Studio, create a UI Policy or open an existing UI Policy for editing.
2. Scroll to the **UI Policy Related List Actions** related list. If creating a new UI Policy, the UI Policy must be saved before the *UI Policy Related List Actions* related list is visible.
3. Click the **New** button.
4. Configure the UI Policy Related List Action.

1. Select a *List* name.
2. Set the *Visible* field value.
3. ***True***: Apply the attribute to the related list.

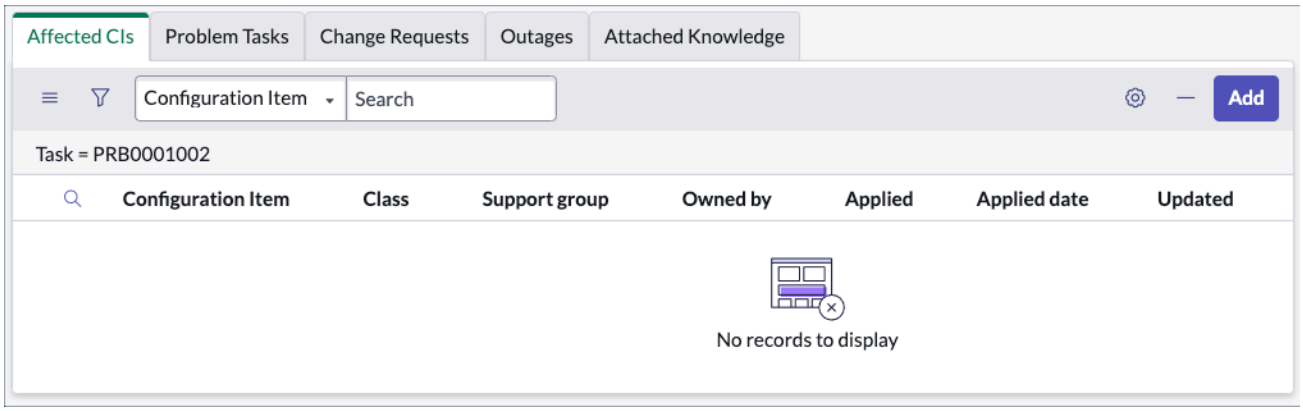
   False: Do not apply the attribute to the related list.

   Leave alone: The attribute does not apply to the related list.

5. Click the **Submit** button.



The *Incidents* related list is hidden:



ARTICLE (15 OF 21)

UI Policy Scripts

UI Policy scripts use the client-side API to execute script logic based on whether the UI Policy condition tests true or false. Use UI Policy scripts to create complex conditional checks or to take actions other than setting field attributes (mandatory, read-only, or visible).

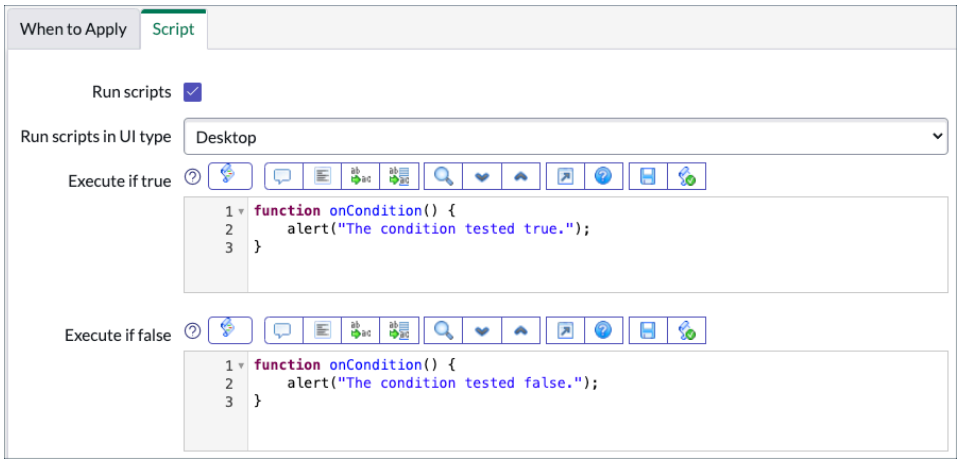
The scripting fields for UI Policies are only visible in the *Advanced* view. To enable the scripting fields, select the *Run scripts* option.

- The *Execute if true* script executes when the UI Policy condition tests *true*.

- The *Execute if false* script executes when the UI Policy condition tests *false*.

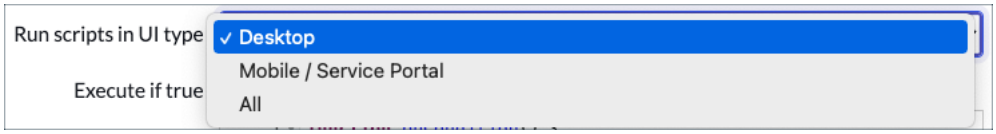
Developers can write scripts in one or both script fields depending on the application requirements.

Write script logic inside the *onCondition* function which is automatically inserted in the scripting fields. The *onCondition()* function is called by the UI Policy at runtime.



IMPORTANT: The *Reverse if false* option must be selected for the *Execute if false* script to run.

Although UI Policy Actions execute on all platforms, UI Policy Scripts execute on Desktop/tablet only in the default case. Use the *Run scripts in UI type* field to select the platforms for the UI Policy scripts.



EXERCISE (16 OF 21)

Exercise: Create UI Policies

In this exercise, you will write, test, and debug a UI Policy.

- Create a UI Policy Action to make the *Other* field visible and mandatory
- Write a script to coach users on what to put in the *Other* field

Preparation

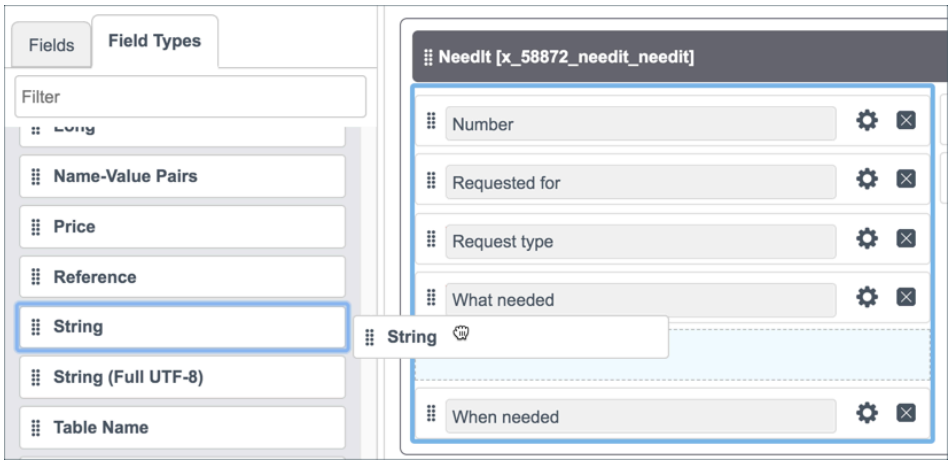
The *What needed* choice list has a value *Other*. When *Other* is selected, users should give more information about what they need by explaining their request in a new field called *Other*.

1. Add a field to the *NeedIt* table.

1. In Studio, use the Application Explorer to open **Forms & UI > Forms > NeedIt [Default view]**.

2. In the Field Navigator, select the **Field Types** tab.

3. Drag the **String** field type and drop it on the NeedIt form between the *What needed* and *When needed* fields.



4. Hover over the *New String* field and click the **Edit Properties** button (

   ⚙

).

5. Configure the Field Properties for the new field:

   1. *Label*: **Other**
      Name: **u_other**

6. Close the *Properties* dialog by clicking the **Close** button (

   ☒

).

2. Click the **Save** button.

Create the UI Policy

1. Create a UI Policy.

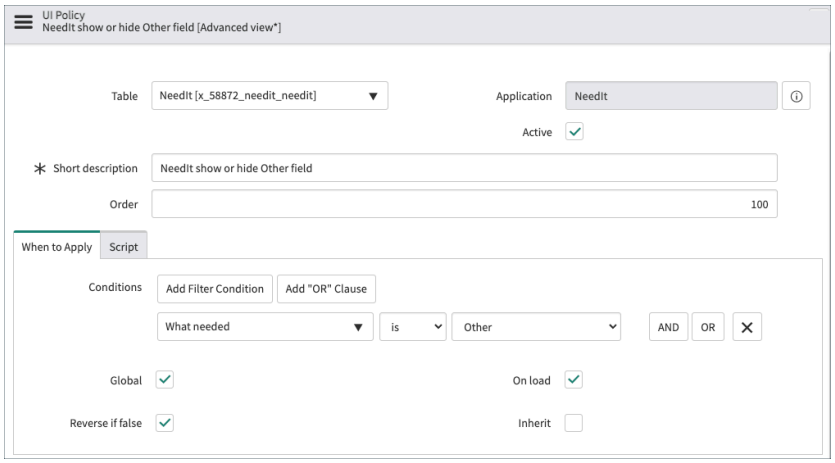
   1. In Studio, click the **Create Application File** link.
   2. In the *Filter...* field enter the text **UI Policy** OR select **Client Development** from the categories in the left hand pane.
   3. Select **UI Policy** in the middle pane as the file type, then click the **Create** button.

2. Configure the UI Policy:

    1. *Table*: **NeedIt [x_<your_company_code>_needit_needit]**
       Active: **Selected (checked)**
       Short Description: **NeedIt show or hide Other field**

3. Switch to the **When to Apply** section and continue configuring the UI Policy:

    1. *Condition*: **[What needed] [is] [Other]**
       Global: **Selected (checked)**
       Reverse if false: **Selected (checked)**
       On load: **Selected (checked)**
       Inherit: **Not selected (not checked)**

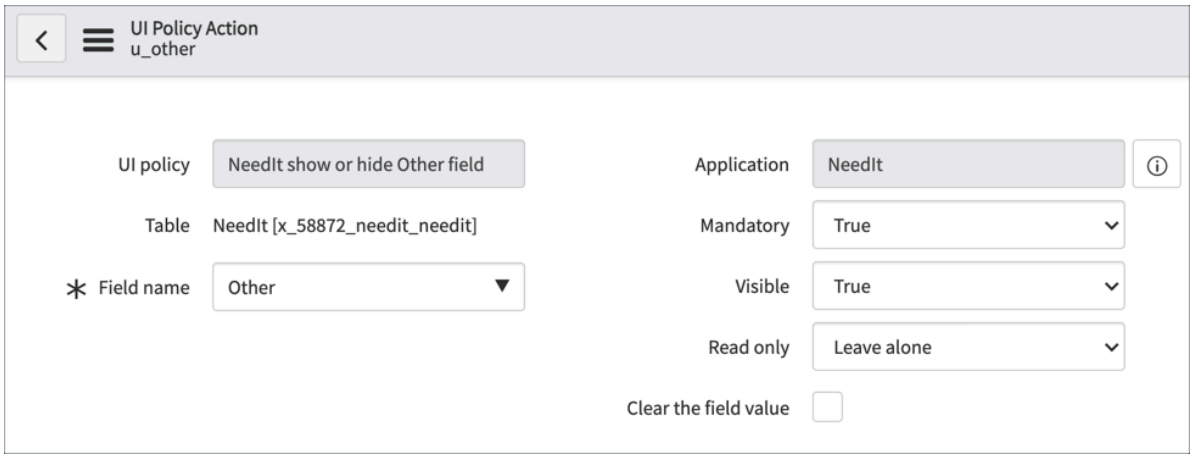


4. Click the **Submit** button.

Create a UI Policy Action

1. In the *NeedIt show or hide Other field* UI Policy, scroll to the **UI Policy Action** section.
2. Click the **New** button.
3. Configure the UI Policy Action.
    1. *Field name*: **Other**
       Mandatory: **True**

Visible: **True**



4. Click the **Submit** button.

Test the UI Policy and UI Policy Action

1. In the main ServiceNow browser window (not Studio), use the browser's reload button to load the latest client-side logic into the browser.
2. Use the *All* menu to open **NeedIt > Create New**.
3. Set the *What needed* field value to **Other**.
4. Does the *Other* field appear? Is it mandatory? If not, debug and re-test.

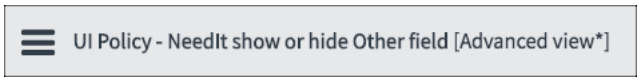
5. Change the *What needed* field value to anything *except* **Other**.
6. Does the *Other* field disappear? If not, debug and re-test.

Create the UI Policy Execute if True Script

1. Return to the Studio tab.

2. If not still open, open the **NeedIt show or hide Other field** UI Policy for editing.

3. Look in the form header to see if the view is the *Advanced view*. If not, click the **Advanced view** Related Link.



4. Switch to the **Script** section (tab).

5. Select the **Run scripts** option (checked).

6. Set the *Run scripts in UI type* value to **All**.

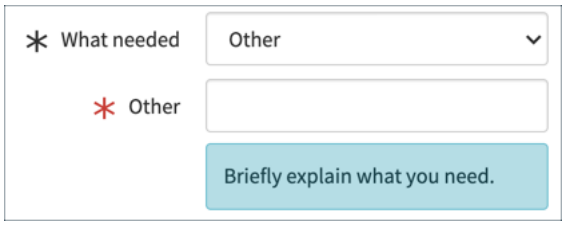
7. Replace the contents of the *Execute if true* script field with this script:

```
function onCondition() {
                    // Display a message under the Other field to explain what to put in the
                    // Other field.
                    g_form.showFieldMsg('u_other','Briefly explain what you need.','info');
          }
```

8. Click the **Update** button.

Test the Execute if True Script

1. In the main ServiceNow browser window (not Studio) use the *All* menu to open **NeedIt > Create New**.
2. Set the *What needed* field value to **Other**.
3. Does the message appear underneath the *Other* field? If not, debug and re-test.



4. Change the *What needed* field value to anything *except* **Other**.
5. Change the *What needed* field value back to **Other**.

Challenge: Create the UI Policy Execute if False Script

Write a script in the *Execute if false* field to clear the message written by the *Execute if true* script. Use the *GlideForm hideFieldMsg()* method. A description of the *hideFieldMsg()* method is available in the API documentation: *GlideForm hideFieldMsg()* documentation (https://developer.servicenow.com/app.do#!/api_doc?v=utah&id=r_GlideFormHideFieldMsg_String_Boolean). Include a comment in your script to explain what the script does.

Test the script to make sure the *Briefly explain what you need.* message is cleared.

ARTICLE (17 OF 21)

Client Scripts vs. UI Policies

Client Scripts and UI Policies both execute client-side logic and use the same API. Both are used to manage forms and their fields. When developing an application, how can you decide which client-side script type to use? Use this table to determine which type is best suited to your application's needs:

Criteria	Client Script	UI Policy
Execute on form load	Yes	Yes
Execute on form save/submit/update	Yes	No
Execute on form field value change	Yes	Yes
Have access to field's old value	Yes	No
Execute after Client Scripts	No	Yes
Set field attributes with no scripting	No	Yes
Require control over order of execution	*Yes	Yes

*Although the *Order* field is not on the Client Script form baseline you can customize the form to add it.

UI Policies execute after Client Scripts. If there is conflicting logic between a Client Script and a UI Policy, the UI Policy logic applies.

EXERCISE (18 OF 21)

Exercise: Save Your Client-side Scripting Work (Optional)

Source control applications, like GitHub, allow developers to commit changes (save completed work) outside of the Personal Developer Instance (PDI). Commit changes made to the application to save your work in source control.

In this exercise, you will save the work completed in this module to your GitHub repository.

> **NOTE**: See the GitHub Guide (/dev.do#!/guide/utah/now-platform/github-guide/github-and-the-developer-site-training-guide-introduction) for more information on how ServiceNow uses GitHub with the Developer Program learning content and to see a video on how to save your work.

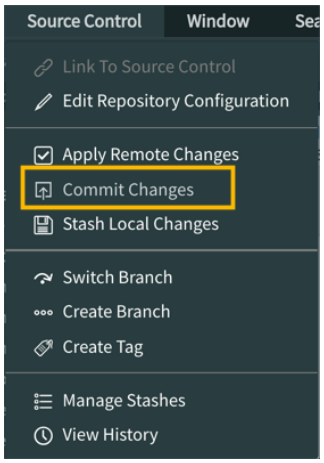
Commit Changes

1. If the *NeedIt* application is not open in Studio, open it now.

    1. In the main ServiceNow browser window, use the *All* menu to open **System Applications > Studio**.

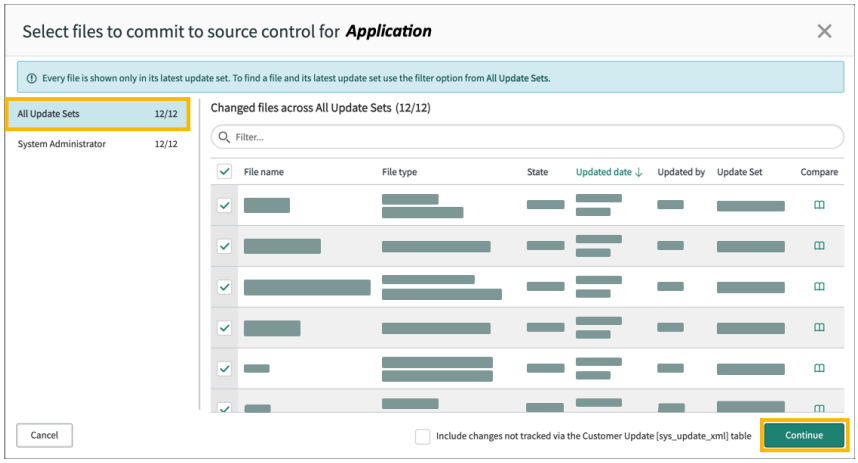
2. In the *Select Application* dialog, click the application.

2. Open the **Source Control** menu and select the **Commit Changes** menu item.



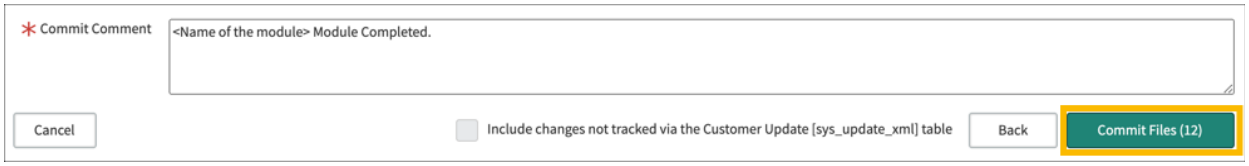
3. Select the updates to commit.

   1. In the *Select files to commit to source control for <Application>* dialog, select **All Update Sets**.
   2. Review the application files to be committed.
   3. Click the **Continue** button.



4. In the *Confirm files to commit to source control for NeedIt* dialog, enter a *Commit comment*, such as **Client-side Scripting Module Completed**.

5. Click the **Commit Files** button.



6. When the *Commit Changes* dialog reports success, click the **Close** button.

NOTE: If the commit change fails, you may have entered the ServiceNow repository URL instead of the forked repository URL in the URL field. See the Troubleshooting GitHub Issues (/dev.do#!/guide/utah/now-platform/github-guide/troubleshooting-github-issues) section in the *GitHub Guide* for instructions on troubleshooting GitHub connection issues.

ARTICLE (19 OF 21)

Test Your Client-side Scripting Knowledge

Want to verify your understanding of client-side scripting? These questions will help you assess your progress. For each question, determine your response then click anywhere in the question to see the answer.

ARTICLE (20 OF 21)

Client-side Scripting Module Recap

Core Concepts:

- Client-side scripts execute script logic in web browsers
- Client-side scripts manage forms and form fields
- Client Scripts execute script logic when forms are:
  - Loaded
  - Changed
  - Submitted/Saved/Updated
- UI Policies have a condition as part of the trigger
- UI Policies can take different actions when conditions return true or false
- UI Policy Actions do not require scripting to set field attributes:
  - *Mandatory*
  - *Visible*
  - *Read only*
- UI Policy Related List Actions do not require scripting to show or hide related lists
- The *GlideForm* API provides methods for interacting with forms and form fields
- The *GlideUser* API provides methods and properties for accessing information about the currently logged in user and their roles

ARTICLE (21 OF 21)

After Completing Client-side Scripting, You Might Also Be Interested In...

Congratulations on completing the Client-side Scripting module. Based on your interest in scripting, you might also enjoy:

- Server-Side Scripting (https://developer.servicenow.com/to.do?u=CSS-U-MOD-SSS): In this Developer Site learning module, you will learn to write, test, and debug server-side scripts using the Now Platform server-side scoped API.
- Data Policies (https://developer.servicenow.com/to.do?u=CSS-U-MOD-DAP): In this Developer Site learning module, you will learn to write, test, and debug Data Polices.
- Client-side Scripting Technical Best Practices (https://developer.servicenow.com/to.do?u=CSS-U-TBP-ClientSideTBP): In this Developer Site Technical Best Practices Guide, you will learn best practices for client-side scripting on the Now Platform.
- Client-side Scripting Documentation (https://developer.servicenow.com/to.do?u=CSS-U-DOC-ClientSideScripting): On the ServiceNow docs site, you will find the complete set of reference material for client-side scripting.